Architecture v1.0 → v2.0

Why We Redesigned the Systolic Array Accelerator

Team Maxilerator | SSCS Chipathon 2026, Track A

What Triggered the Redesign

A deliberate multiplier choice, confirmed by real synthesis

The Decision

We evaluated multiple multiplier architectures for the GF180MCU PDK specifically, and selected a **radix-2 Booth multiplier** as the optimal choice for this process.

This was a deliberate engineering choice — not a default or a synthesis-tool side effect.

The Finding

Once synthesized, the **actual Booth-based mac_unit area came in higher than planned**, for this PDK.

Scaled across all 64 MAC units in systolic_array, this gap consumed the margin the original plan depended on.

This is the direct trigger for the v2.0 redesign.

Architecture at a Glance

v1.0

```
chip_top
 └─ chip_core
     └─ host_interface
         └─ accelerator_core
             ├─ memory
             │   ├─ SRAM_0 (ping)
             │   └─ SRAM_1 (pong)
             ├─ feeder
             ├─ controller (FSM)
             ├─ systolic_array
             │   └─ mac_unit x64
             └─ output_processor
```

v2.0

```
chip_top
 └─ chip_core
     └─ host_interface
         └─ accelerator_core
             ├─ feeder
             │   (streaming, self-drains)
             ├─ systolic_array
             │   └─ mac_unit x64
             │       └─ booth_multiplier
             └─ output_processor
                 (self-generates done)
```

What We Removed — and Why

Two SRAM macros (ping-pong)

18.6% of Slot A combined. Purely a buffer -- valid only because of single-port SRAM read/write conflicts. Removing them requires the host to send data in feeder-friendly order instead.

Controller (FSM)

Centralized 3-state coordinator (IDLE / PROCESSING / OUTPUT). Its job -- deciding when to drain, swap, and reset -- is now handled by the modules that already have that information.

Fixed tile-based transfer (128-byte tiles)

Data now streams continuously, byte by byte, over two independent channels -- no host-side tile buffering or tile_done bookkeeping required.

How Coordination Changed

From one central FSM to modules that self-generate their own phase transitions

v1.0 — Controller-Driven

controller watches tile_done and last_pass, then explicitly drives:

- `write_addr, swap  (to memory)`
- `start  (to feeder)`
- `drain_en, clear, output_en`

Every phase transition passes through one central FSM.

v2.0 — Self-Generated

feeder detects a_last/b_last itself and self-triggers its own drain -> `drain_done`

`drain_done` feeds directly into **output_processor**

output_processor self-generates `output_done` the instant its own final result transfers, and that pulse resets both feeder and the MAC array directly.

No central FSM. No module waits on a coordinator that isn't itself.

Benefits Gained From the Rework

Not just a fix — a genuine upgrade

~2x Throughput

Independent A/B channels allow simultaneous activation and weight delivery — v1.0's single-port SRAM made A/B access inherently sequential.

DATA_WIDTH Flexibility

Supports INT4, 8, 16, 32, 64 — no longer locked to 8-bit by the SRAM macro's fixed width.

Quantization Flexibility

SHIFT_BITS / SAT_MAX / SAT_MIN give calibrated quantization control that v1.0's fixed INT8 saturation didn't have — no RTL changes needed.

Simpler Protocol

No tile_done / last_pass / address-counter bookkeeping — fewer modules, fewer signals, a smaller verification surface.

Cleaner Reusable Interface

accelerator_core's native ports are uniform per-channel data/valid/ready/last — a more natural fit for SoC reuse than the tile-oriented v1.0 interface.

Reviewer's Critical Items — Now Resolved

Conditional Go review, 9.75/15 — three CRITICAL items held it back from a clear Go

Then:	Now:
accelerator_core did not elaborate — systolic_array port convention needed reconciling	Entire hierarchy rewritten cleanly for v2.0; accelerator_core elaborates and simulates end to end.
Then:	**Now:**
LibreLane still built the template chip — chip_core.sv was the wafer-space counter, not the accelerator	chip_core now instantiates host_interface -> accelerator_core directly; config.yaml points at the real RTL hierarchy.
Then:	**Now:**
No end-to-end result on record — CI ran the counter testbench, not the accelerator	tb_accelerator_core: 8/8 passing (single/multi-pass, saturation, backpressure, stalls). chip_top_tb: 2/2 passing through the REAL physical pad interface.

Verification: Then vs Now

v1.0

controller — 13/13
memory — 12/12
feeder — 15/15
output_processor — 12/12
mac_unit — 10/10
systolic_array — 10/10

accelerator_core: not elaborating
chip_top_tb: template counter only

v2.0

feeder — 40/40
output_processor — 28/28
mac_unit — 13/13
systolic_array — 9/9

accelerator_core — 8/8
chip_top (real pads) — 2/2

Net Effect on Area Budget

Real synthesized area, not estimates

v1.0 Total

1,110,253 µm²

878,016 µm² std cells
+ 2 x 116,119 µm² SRAM macros

v2.0 Total

778,769 µm²

Standard cells only — no SRAM, no controller

331,484 µm² reduction — 29.86% smaller

Both totals from real Yosys synthesis, not textbook estimates.

Where We Stand Now

- ✓ Full RTL rewrite: no-SRAM streaming feeder, no-controller self-generated coordination
- ✓ Every module re-verified: feeder (40), output_processor (28), mac_unit(13), systolic_array(9), accelerator_core (8), chip_top (2)
- ✓ Pad-level integration verified through real GF180MCU I/O pad models, not just accelerator_core in isolation
- ✓ chip_core now builds the real accelerator, not the template counter
- ✓ Synthesis complete — real area confirmed (778,769 µm², 29.86% smaller than v1.0)
- ✓ Dry-run GDS submission complete
- ✓ GDS generated — DRC, LVS, and antenna checks all clean

Working through setup timing violations at slower process corners — the one open item before final closure.